



Otonom Uzay Trafik Yönetim Sistemi

Çarpışma Önleme, Manevra Optimizasyonu ve Yer Segmenti Planlaması için Ar-Ge Prototipi

Şebnem Başak

Yazılım Mühendisi

Haziran 2026

Özet

ASTM (Otonom Uzak Trafik Yönetimi), Alçak Dünya Yörüngesi'ndeki (LEO) uydular arasındaki çarpışma risklerini tespit eden, optimal kaçınma manevraları hesaplayan ve büyüyen uyduların yer istasyonu kapasite ihtiyaçlarını planlayan bağımsız bir Ar-Ge prototipidir.

Proje Türkiye Uzay Ajansı'nın Milli Uzay Programı'nda belirtilen "Uzay Nesnelerinin Yerden Gözlemi ve Takibi" (Hedef 7) stratejisi doğrultusunda, ülkenin uzay trafik yönetimi alanındaki yazılım kabiliyetini artırmak amacıyla tasarlandı. İlk sürümü Aralık 2025'te yayınlandı, Haziran 2026'da yeni modüllerle (manevra tespiti, gelişmiş makine öğrenmesi, yer istasyonu kapasite planlaması) genişletildi.

Sistemin doğruluğu Uluslararası Uzay İstasyonu'nun (ISS) iki gerçek modülü (ZARYA ve NAUKA) üzerinde test edilmiş, kenetlenme durumunu 5 metrenin altında hassasiyetle tespit etmiş ve bu sonuç, proje kod tabanından tamamen bağımsız ikinci bir kütüphaneyle çapraz doğrulanmıştır.

Bu doküman sistemin mimarisini, kullanılan algoritmaları, gerçek dünya doğrulamasını, geliştirme sürecinde karşılaşılan ve çözülen mühendislik sorunlarını ve projenin güncel durumunu sunuyor.

1. Problem: Uzayda Artan Trafik Yoğunluğu

1.1 Kessler Sendromu ve Yörünge Güvenliği

1978'de NASA bilim insanı Donald Kessler'ın öngördüğü senaryo, bugün giderek daha somut bir risk haline geliyor. Yörüngedeki enkaz yoğunluğu belirli bir eşiği aştığında, her çarpışma yeni enkaz üretiyor. Yeni enkaz daha fazla çarpışmaya yol açıyor ve bu süreç zincirleme bir reaksiyona dönüşüyor.

Bu senaryo artık teorik değil. 2009'da Iridium 33 ile Kosmos 2251'in çarpışması, 1500'den fazla takip edilebilir enkaz parçası üretti. 2021'de Rusya'nın anti-uydu testi, ISS mürettebatını kurtarma kapsüllerine sığınmak zorunda bıraktı. Yörüngedeki nesne sayısı, sadece operasyonel uydular değil, izlenemeyen küçük enkaz da dahil edildiğinde on binlerle ifade ediliyor.

Uzay trafik yönetimi (Space Traffic Management / STM), bu ortamda kritik bir altyapı ihtiyacı haline geldi. Hangi nesnelerin birbirine yaklaşacağını önceden tespit etmek, tehlikeli yaklaşımlara müdahale planlamak ve artan uydular sayısına rağmen sistemi ölçeklenebilir tutmak.

1.2 Hesaplama Yüğü ve Ölçeklenebilirlik Sorunu

N adet nesnenin tüm ikili kombinasyonları kontrol edildiğinde, $N(N-1)/2$ işlem gerekir ki bu $O(N^2)$ karmaşıklığıdır. 1000 uydular için bu her bir zaman anında yaklaşık 500.000 karşılaştırma demektir. Uydular sayısı 10.000'e yaklaştığında, aynı yaklaşımla saniyede 50 milyon karşılaştırma yapmak gerekir. Bu yalnızca yazılım verimliliğiyle değil, doğru veri yapısı ve algoritma seçimiyle aşılabilecek bir problemdir.

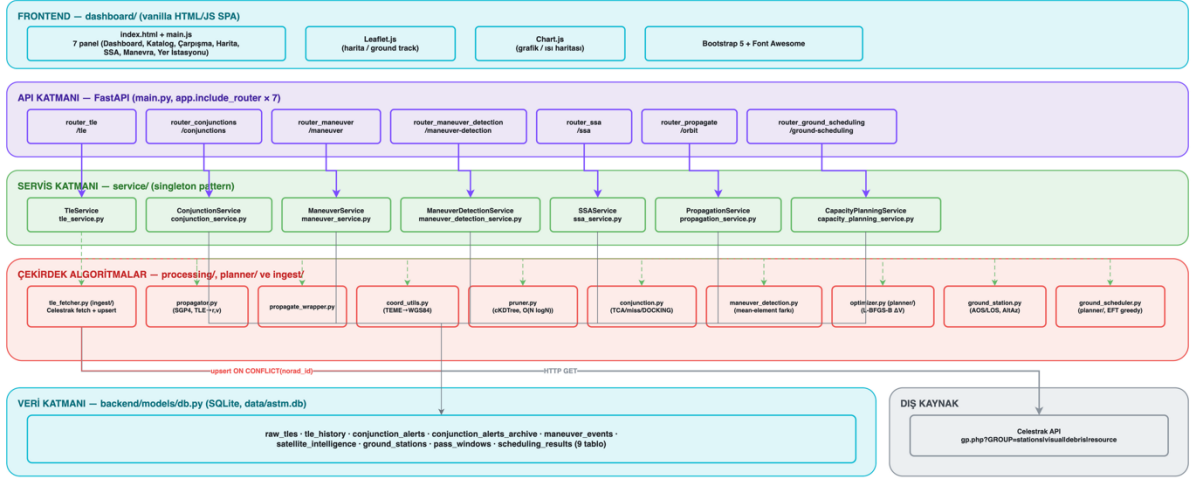
ASTM, bu problemi üç ayrı ama birbirine bağımlı alt-problem olarak ele alıyor: (1) hangi uydular çiftlerinin gerçekten risk taşıdığını verimli şekilde bulmak, (2) risk tespit edildiğinde en az yakıtla güvenli bir kaçınma manevrası hesaplamak ve (3) büyüyen uyduların yer segmenti kapasitesini önceden planlamak.

2. Sistem Mimarisi

Sistem, servis katmanı mimarisi ile tasarlandı. API katmanı (FastAPI), iş mantığı katmanı (servisler) ve çekirdek hesaplama katmanı (uzay mekaniği algoritmaları) birbirinden net şekilde ayrılıyor. Bu ayırım, her katmanın bağımsız olarak test edilebilmesini ve yeni modüllerin (örneğin sonradan eklenen manevra tespiti veya yer istasyonu planlaması) mevcut yapıyı bozmadan eklenebilmesini sağladı.

ASTM Prototip — Katmanlı Sistem Mimarisi (v2.0.0)

main.py → 7 router → 7 servis → processing/planner → SQLite (9 tablo) | Celestrak → ingestile_fetcher.py → raw_files



Şekil 1: Sistem mimarisi — frontend, FastAPI backend, servis katmanı, çekirdek algoritmalar ve veri katmanı arasındaki veri akışı.

Backend, hesaplama yoğunluklu uzay mekaniği görevlerini yöneten FastAPI üzerine kurulu. Frontend ise etkileşimli bir kontrol paneli, uydu kataloğu ve Leaflet.js tabanlı canlı harita görselleştirmesi sunan bir HTML/JavaScript arayüzü. Veri katmanında SQLite kullanılıyor. Güncel şema 9 tabloyu kapsıyor (ham TLE kayıtları, çarpışma uyarıları ve arşivi, TLE geçmişi, manevra olayları, uydu zekası tahminleri, yer istasyonları, geçiş pencereleri ve çizelgeleme sonuçları).

3. KD-Tree ile Ölçeklenebilir Çarpışma Tarama

3.1 Yörünge Yayılımı SGP4 Modeli

Bir uydunun gelecekteki konumunu hesaplamak için TLE (Two-Line Element) verisi tek başına yeterli değildir çünkü veri yalnızca belirli bir andaki yörünge elemanlarını içerir. Zamanın bir fonksiyonu olarak $r(t)$ (konum) ve $v(t)$ (hız) vektörlerine ulaşmak için bir propagasyon modeli gerekir.

ASTM'de SGP4 (Simplified General Perturbations-4) modeli kullanılmaktadır. SGP4, yalnızca ideal çift-cisim dinamiğini değil, gerçek dünya perturbasyonlarını da hesaba katar. Dünya'nın tam küresel olmamasından kaynaklanan J2 etkisi, atmosferik sürüklenme, Güneş ve Ay'ın kütle çekim etkileri. Bu perturbasyonlar, özellikle alçak yörüngede (LEO) yörünge tahminini birkaç dakika içinde kilometre mertebesinde saptırabilir. Yüksek hassasiyetli çarpışma analizi için hesaba katılmaları zorunludur.

SGP4 çıktısı TEME (True Equator, Mean Equinox) koordinat sistemindedir. TEME Dünya merkezli eylemsiz bir koordinat sistemidir. Harita görselleştirmesi için gerekli Enlem/Boylam/İrtifa formatına dönüşüm, Dünya'nın dönüşü (Greenwich Sidereal Time) hesaba katılarak astropy kütüphanesi aracılığıyla yapılmaktadır.

3.2 KD-Tree ile $O(N \log N)$ Budama

N nesnenin tüm ikili kombinasyonları incelenirse $O(N^2)$ karmaşıklığa ulaşılır. 1000 uydu için her bir anda 500.000 karşılaştırma, 10.000 uydu için 50 milyon karşılaştırma. Bu karmaşıklık gerçek zamanlı bir sistemde sürdürülebilir değildir.

ASTM bu problemi uzamsal indekisleme (KD-Tree) ile çözüyor. Telefon rehberinde bir ismi tüm listeyi taramadan, alfabetik sıraya göre hızla bulmak nasıl mümkünse, KD-Tree de üç boyutlu uzayda "bu noktanın 300 km içinde hangi diğer noktalar var?" sorusunu $O(\log N)$ karmaşıklıkla yanıtlar.

500.000 ikili karşılaştırma yerine, yalnızca gerçekten risk potansiyeli taşıyan birkaç yüz aday çift detaylı analize gönderiliyor. Bu budama (broad phase) tek bir anlık görüntüde çalışır. Sınırlamaları ve önerilen düzeltme yöntemi KNOWN_LIMITATIONS.md'de belgelenmiştir.

3.3 İki Aşamalı Budama: Analitik TCA + SGP4 Refinement

Aday çiftler belirlendikten sonra iki aşamalı bir analiz uygulanıyor:

- Analitik Filtre (Lineer Varsayım): Uyduların kısa süre doğrusal hareket ettiği varsayımıyla En Yakın Yaklaşma Zamanı (TCA) kapalı form olarak hesaplanır:

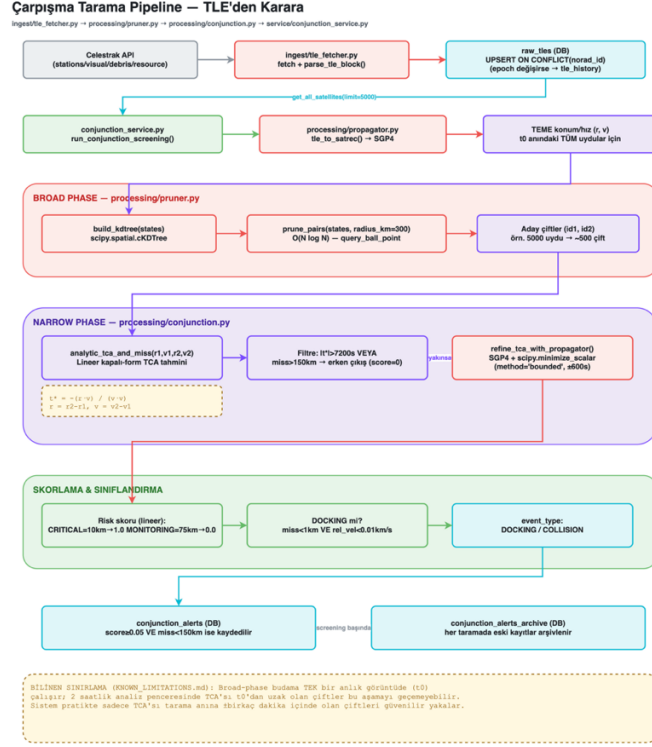
$$t^* = -(r \cdot v) / (v \cdot v)$$

r: bağıl konum vektörü (km)

v: bağıl hız vektörü (km/s)

- SGP4 İyileştirmesi (Refinement): Analitik yöntem 5-10 saniyelik hatalar içerebilir. Eğer analitik mesafe yeterince küçükse, t^* civarında `scipy.optimize.minimize_scalar` ile gerçek SGP4 propagasyonu kullanılarak kesin minimum mesafe (Miss Distance) hesaplanır. Yörüngenin eğriliği (curvature) bu aşamada dikkate alınır.

Bu iki-aşamalı yaklaşım, pahalı hesaplamanın yalnızca gerçekten gerekli olduğu yerde çalışmasını sağlıyor. Sonuç mesafe ve görelî hıza göre COLLISION (çarpışma riski) veya DOCKING (kasıtlı kenetlenme/formasyon uçuşu) olarak sınıflandırılıyor. Bu ayrım yanlış alarm üretmeden gerçek tehlikeleri tespit edebilmek için kritiktir (bkz. Bölüm 5, ISS doğrulama testi).

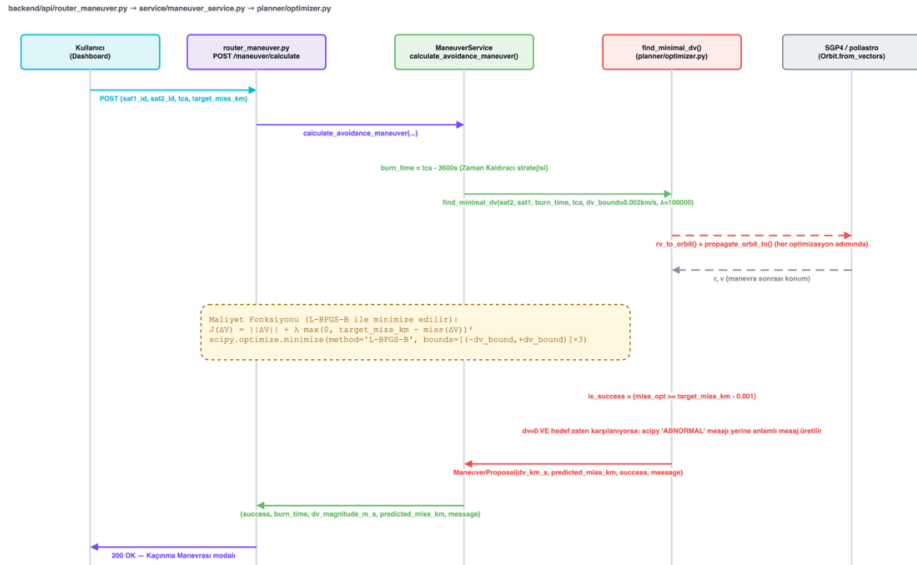


Şekil 2: Çarpışma tarama pipeline'ı — TLE verisinden KD-Tree broad-phase budamaya, analitik TCA tahmininden SGP4 refinement ve risk skorlamaya kadar veri akışı.

4. Manevra Optimizasyonu

Bir çarpışma riski tespit edildiğinde, sistem yalnızca uyarı vermiyor aynı zamanda optimal bir kaçınma planı hesaplıyor. Buradaki soru şu: motorları şu an ateşleyip hız vektörüne belirli bir değişim yaparsak, çarpışma anında (TCA) diğer nesneye ne kadar uzak oluruz ve bunu en az yakıtle nasıl başarırız?

Kaçınma Manevrası Optimizasyonu — Sequence Diyagramı



Şekil 3: Manevra optimizasyonu akış diyagramı — kullanıcı isteğinden, optimizasyon döngüsü üzerinden nihai ΔV vektörüne kadar.

Manevra, çarpışmadan belirli bir süre önce (üretimde 1 saat) uygulanan ani (impulsive) bir hız değişimi olarak modelleniyor. poliastro kütüphanesi kullanılarak, bu hız değişiminin sonucunda oluşan yeni yörünge iki-cisim problemi (Keplerian propagation) olarak çözülüp TCA anına kadar ilerletiliyor.

Optimizasyon, scipy'nin L-BFGS-B (kutu kısıtlı, gradyan tabanlı) algoritmasıyla aşağıdaki maliyet fonksiyonunu minimize ediyor:

```
J(ΔV) = |ΔV| + λ · max(0, HedefMesafe - Mesafe(ΔV))2

|ΔV|      : harcanan yakıt miktarı (minimize edilmek isteniyor)
λ (lambda) : ceza katsayısı – güvenlik, yakıttan daha önemli
max(...)² : hedef mesafeye ulaşamazsa büyük ceza puanı
```

Birinci terim yakıt maliyetini, ikinci terim ise hedef mesafeye ulaşamaması durumunu cezalandırıyor. λ katsayısı güvenliği yakıtın önüne koyuyor. Mesafe yeterliyse penalty sıfır, değilse hızla büyüyen bir ceza ekleniyor. Bu formülasyon, 'güvenli mesafeyi sağlayan en küçük ΔV değerini bul' ilkesini matematiksel olarak karşılıyor.

Manevra, çarpışmadan TCA'dan 1 saat önce uygulandığında özellikle etkili. Erken yapılan küçük bir açı değişikliği (Time Leverage prensibi), TCA anında büyük bir konum farkı yaratır. Yörünge mekaniğinin doğrusal olmayan yapısı bu avantajı sağlar.

5. Gerçek Dünya Doğrulaması

Geliştirilen sistemin doğruluğu, soyut bir test senaryosuyla değil, gerçek bir uzay aracı verisiyle doğrulandı. Uluslararası Uzay İstasyonu'nun iki modülü (ISS (ZARYA), NORAD ID 25544, ve ISS (NAUKA), NORAD ID 49044) birbirine fiziksel olarak kenetli durumdayken, 1 Aralık 2025 tarihli gerçek TLE verileriyle sisteme verildi.

5.1 Sonuçlar

- Tespit edilen en yakın yaklaşma zamanı (TCA): 2025-12-02T05:44:53 UTC
- Hesaplanan minimum mesafe: 0.0054 km (yaklaşık 5,4 metre)
- Sınıflandırma: Sistem bu durumu doğru şekilde DOCKING (kenetlenme) olarak işaretledi, COLLISION (çarpışma) uyarısı vermedi

5.2 Bağımsız Doğrulama

Bu sonucu tek bir kütüphaneye güvenerek bırakmadım. Proje kod tabanından tamamen ayrı, yalnızca skyfield kütüphanesini kullanan bağımsız bir doğrulama betiği yazıp aynı TLE verileriyle çalıştırdım. Bağımsız betik, TCA civarında 1 saniyelik adımlarla tarama yaparak minimum mesafeyi metre-altı hassasiyetle buldu. Sonuç, ana sistemin ürettiği değerle tutarlıydı.

6. Uzak Durum Farkındalığı (SSA) ve Makine Öğrenmesi

ASTM, ham TLE verilerinin ötesine geçerek uyduların olası görevini, davranış normalliğini ve atmosfere düşme riskini tahmin eden bir makine öğrenmesi katmanı içeriyor. Bu katman, Union of Concerned Scientists (UCS) Uydu Veritabanı'ndan (7.500'den fazla uydu) eğitiliyor.

6.1 Görev Sınıflandırma

200 ağaçlı bir Random Forest sınıflandırıcısı, yörünge parametrelerinden (eğim, dış merkezlik, periyot, perije, apoje) uydunun olası görev tipini tahmin ediyor. Ham olasılık çıktıları, ağaç tabanlı modellerin genellikle aşırı güvenli (overconfident) tahminler üretmesi nedeniyle, çapraz doğrulamalı sigmoid kalibrasyon (CalibratedClassifierCV) ile düzeltiliyor. Bu kalibrasyon doğruluğu %89.3'ten %90.2'ye, ROC-AUC'u 0.979'dan 0.984'e yükseltti.

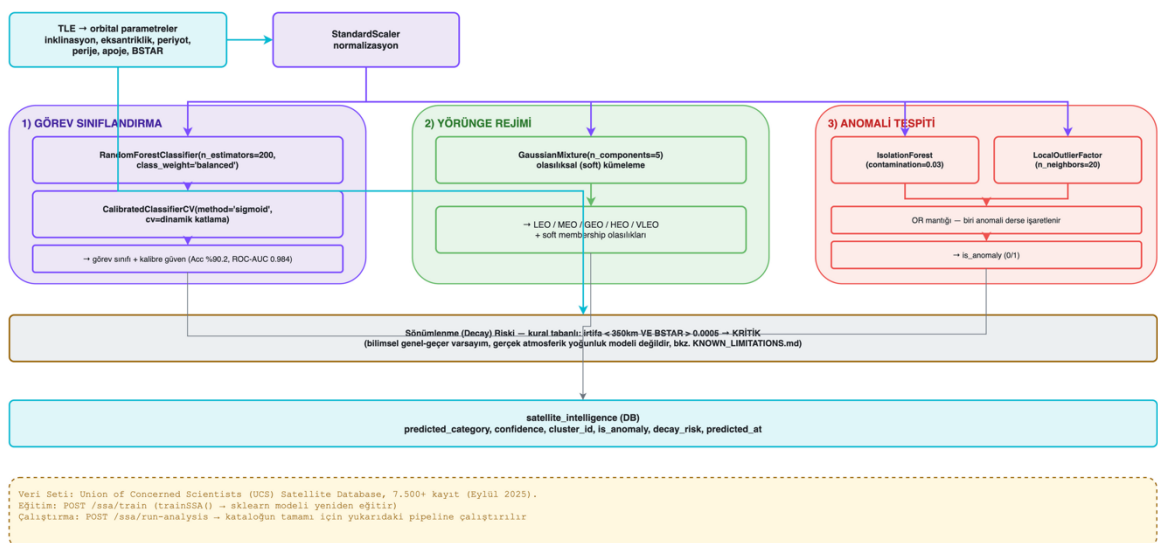
6.2 Yörünge Rejimi Kümeleme

Uydular, K-Means yerine Gaussian Mixture Model (GMM) ile 5 ana yörünge rejimine (LEO, MEO, GEO, HEO, VLEO) kümelenebilir. Bu tercih bilinçli. K-Means'in sert (hard) küme sınırları yerine, rejim sınırındaki uydular için olasılıksal bir küme ataması sağlıyor çünkü gerçek dünyada yörünge rejimleri arasındaki geçişler de keskin değil, kademeli.

6.3 Anomali Tespiti

Isolation Forest ve Local Outlier Factor (LOF) algoritmaları bir araya getirilerek bir ensemble oluşturuldu. Isolation Forest global/eksen-hizalı aykırılıkları yakalarken, LOF yoğun bir kümenin içindeki lokal anormallikleri tespit ediyor. İki yöntemden biri bile anomali derse nesne işaretleniyor. Kaçırılan gerçek bir anomalinin maliyeti, fazladan bir uyarının maliyetinden yüksek olduğu için bilinçli bir tasarım tercihi. 170 canlı uydu üzerinde yapılan doğrulamada, bu ensemble tek başına Isolation Forest'a göre %13'e varan (önceki %3'ten) bir tespit oranına ulaştı.

SSA / Yapay Zeka Pipeline — service/ssa_service.py (SSAService)



Şekil 4: SSA / Yapay Zeka Pipeline — TLE parametrelerinden görev sınıflandırma, yörünge rejimi kümeleme ve anomali tespitine kadar üç paralel ML katmanı.

7. Geçmiş Manevra Tespiti

Bu modül, bir uydunun TLE geçmişinden, daha önce gerçekleştirdiği yörünge manevralarını geriye dönük olarak tespit ediyor. Yörünge mekaniğinde sezgisel olmayan ama önemli bir analizi içeriyor.

İlk yaklaşım terk edildi

İlk tasarım bir TLE'yi SGP4 ile bir sonraki TLE'nin epoch'una ilerletip, tahmin edilen hız ile gerçek hızı karşılaştırmaktı (artık-hız/residual velocity yöntemi). Bu yöntem gerçek verilerde anlamsız sonuçlar üretti. 54 saat arayla alınmış, aralarında hiçbir manevra olmayan gerçek bir ISS TLE çiftinde, bu yöntem 3.58 m/s'lik yanlış bir “manevra” sinyali hesapladı.

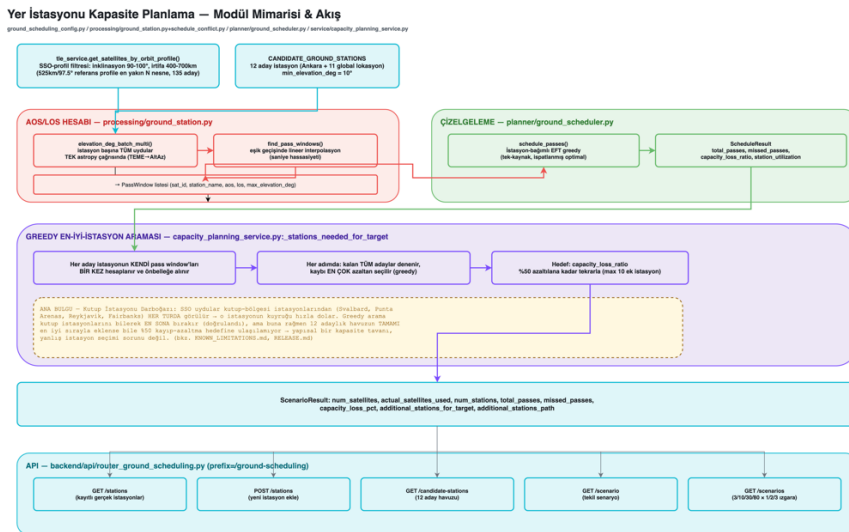
Düzeltilme: Mean-element farkı

Bunun yerine, ardışık iki TLE'nin mean orbital elementleri (yarı-büyük eksen, eğim, dış merkezlik) hiçbir propagasyon yapılmadan doğrudan karşılaştırılıyor. Bu elementler uydunun yörüngedeki anlık fazından (mean anomaly) bağımsız olduğu için, saatler veya günler arayla alınmış TLE çiftlerinde de güvenilir sonuç veriyor. Δa 'dan tanjantsal (irtifa) ΔV , Δi 'den normal (düzlem değişimi) ΔV türetilip toplam manevra büyüklüğü ve tipi (irtifa değişimi, düzlem değişimi, dış merkezlik değişimi veya kombine) sınıflandırılıyor.

8. Yer İstasyonu Planlaması ve Kapasite Analizi

En yeni modül büyüyen bir uydu filosunun (örneğin 3'ten 80'e çıkan bir constellation), sınırlı sayıda yer istasyonu ile karşılaşacağı kapasite darboğazlarını modelliyor. SGP4 görüş geometrisi hesabı (AOS/LOS), istasyon-bağımlı çizelgeleme (Earliest-Finish-Time greedy algoritması) ve greedy en-iyi-istasyon seçimi bir araya getirilerek, “kaç istasyon yeterli” sorusuna bir cevap üretiliyor.

Bu modülün performans optimizasyonu üzerine ekstra çalışmalar yapıldı. İlk implementasyonda tam senaryo taraması ~25-30 dakika sürüyordu. İstasyon başına tüm uyduların konumlarını tek bir astropy çağrısında toplu işleyen bir yöntemle bu süre ~4 dakikaya indirildi (~7 kat hızlanma), sonuçların doğruluğundan ödün verilmeden (eski ve yeni yöntem sonuçları bit-bit karşılaştırıldı).



Şekil 5: Yer istasyonu kapasite planlama modülü — TLE yayılımı ve AOS/LOS hesabından greedy istasyon seçimine ve API katmanına kadar veri akışı.

9. Bakım Etki Analizi

Büyüyen bir yer istasyonu ağında, planlı bir bakım çalışması (anten söküm, yazılım güncellemesi, donanım değişimi) antenin belirli bir süre çevrimdışı kalmasını gerektirir. Bu modül şu soruya cevap veriyor: bakım hangi zaman aralığında yapılırsa en az sayıda, en az öncelikli uydu geçişi kaybedilir?

9.1 Yörünge Ömrü Tahmini ve Ağırlıklandırma

Her uydunun kaybedilen bir geçişinin operasyonel maliyeti aynı değildir. Atmosfere yakın zamanda düşecek bir uydunun geçişini kaçırmak, ömrü yıllarca olan bir uydununkini kaçırmaktan daha kritiktir. Bu nedenle her uyduya, TLE'deki B^* atmosferik sürüklenme katsayısından türetilen bir "kayıp önceliği" ağırlığı atanıyor.

Yörünge ömrü, 1976 US Standard Atmosphere piecewise-exponential atmosfer modeli (150–1000 km irtifa aralığı, Vallado Tablo 8-4) kullanılarak B^* 'tan türetiliyor:

$$T_{\text{ömür}} = 365 \times (\rho_{\text{KAL}} / \rho(h)) \times (B^*_{\text{KAL}} / B^*)$$

$\rho(h)$: h irtifasındaki atmosfer yoğunluğu

Kalibrasyon çıpası: 400 km irtifa, $B^* = 1 \times 10^{-4}$, 365 gün referans ömür

Tahmini ömre göre ağırlık basamaklandırılıyor: 180 günden kısa ömür → ağırlık 3.0, 180-365 gün → 2.0, 365 günden uzun → 1.0. B^* verisi eksikse ($B^* = 0$) nötr ağırlık (1.0) uygulanıyor.

9.2 B^* Regresyonu (Zaman İçinde Bozunma Trendi)

B^* tek bir TLE anlık görüntüsünden okunduğunda ölçüm gürültüsüne karşı hassastır. Bunun yerine, tle_history tablosunda arşivlenen geçmiş TLE kayıtlarına doğrusal regresyon uygulanarak B^* 'ın zaman içindeki trendi hesaplanıyor (en az 3 geçmiş kaydolması koşuluyla, yetersizse tek TLE'deki anlık değere düşülüyor). B^* 'ın yükselme eğilimi göstermesi, uydunun beklenenden hızlı bozulduğuna işaret eder ve ağırlığı buna göre artırır. Bu, sadece "şu an nerede" değil "ne yönde ilerliyor" bilgisini modele katıyor.

9.3 Pencere Skorlama

Sistem, bakımın planlanabileceği 7 günlük bir ufku 30 dakikalık adımlarla tarayarak yüzlerce aday bakım penceresi üretiyor. Her aday pencere için bir maliyet puanı hesaplanıyor:

$$\text{maliyet} = \sum (\text{kaybedilen_geçiş.süresi} \times \text{uydu.ağırlığı})$$

Toplam yalnızca, aday pencereyle zaman olarak çakışan geçişler (AOS'u pencere bitişinden önce ve LOS'u pencere başlangıcından sonra olan geçişler) üzerinden alınıyor. Sonuçta en düşük maliyetli pencereler "önerilen bakım aralıkları", en yüksek maliyetliler "kaçınılması gereken aralıklar" olarak sıralanıyor.

9.4 Doğrulama

Modül, gerçek Celestrak TLE verisiyle uçtan uca test edildi. Örnek bir istasyon için 4 saatlik bir bakım senaryosunda, 7 günlük ufukta ~450 gerçek geçiş, ~2700–2800 dakika toplam temas süresi ve ~330 aday pencere değerlendirildi; en iyi aralık sıfır kayıpla işaretlendi.

10. Geliştirme Sürecinde Düzeltme Geçmişi

Bu sistemin güvenilirliği hiç hata yapmamış olmasından değil, hataların sistematik olarak bulunup belgelenerek düzeltilmiş olmasından geliyor. Aşağıda geliştirme sürecinde tespit edilip çözülen başlıca sorunlardan bir seçki yer alıyor.

Veri bütünlüğü sorunları

- Çarpışma uyarıları her taramada tamamen siliniyordu, geçmiş kayıp gidiyordu. Arşiv tablosu eklenerek geçmiş korunur hale getirildi.
- BSTAR sürüklenme katsayısı için yazılan özel ayrıştırıcı, negatif/sıfır değerlerde hatalı sonuç veriyordu. Standart kütüphane fonksiyonuyla değiştirildi.

Algoritma doğruluğu sorunları

- SGP4 propagasyon fonksiyonu hız vektörünü atıyordu, bu da gereksiz yere iki kat hesaplama yapılmasına neden oluyordu. Düzeltip hem performans hem doğruluk kazanıldı.
- Makine öğrenmesi özellik vektöründe irtifa alanı yanlışlıkla iki kez yazılıyordu, gerçek perije/apoje değerleri modele doğru verilmeyordu. Düzeltme sonrası yüksek-eksantriklikli uydularda tahmin doğruluğu arttı.
- Manevra tespiti için ilk denenen yöntem (artık-hız karşılaştırması), gerçek veride fiziksel olarak anlamsız sonuçlar üretti. Mean-element farkı yöntemine geçilerek çözüldü (bkz. Bölüm 7).

Performans sorunları

- Yer istasyonu kapasite taramasının ~25-30 dakika sürmesi, koordinat dönüşüm çağrılarının toplu işlenmesiyle ~4 dakikaya indirildi.
- Harita üzerindeki canlı uydu takibi, hesaplama penceresi sonunda donuyordu. Pencere sonuna yaklaşıldığında arka planda otomatik yenileme eklendi.

Güvenlik ve yapılandırma

- CORS politikası tüm originlere açıldı. Ortam değişkeninden okunan, yapılandırılabilir bir politikaya çevrildi.

Bu liste seçicidir; sürüm notlarının (RELEASE.md) tamamı, her düzeltme için tarih, commit referansı ve teknik gerekçeyi içerir.

11. Test Kapsamı ve Teknik Ölçekler

Metrik	Değer
Toplam otomatik test	36 (tamamı geçiyor)
Veritabanı tabloları	9
Ana modül sayısı	6 (çarpışma, manevra optimizasyonu, SSA/ML, manevra tespiti, yer istasyonu planlaması, veri katmanı)
İlk sürüm	Aralık 2025
Güncel sürüm	v2.0.0 (Haziran 2026)

Tablo 1: Projenin güncel teknik kapsamı.

Çekirdek uzay mekaniği modülleri, gerçek SGP4/poliastro çağrılarını yerine doğrusal hareket varsayan deterministik sahte propagatör fonksiyonlarıyla test ediliyor. Bu da testlerin sonuçlarının doğrulanabilir değerlerle karşılaştırılabilmesini sağlıyor. Yani “test geçti” ifadesi, gerçek fiziksel doğrulukla karıştırılmıyor. Fiziksel doğruluk ayrı olarak (bkz. Bölüm 5) gerçek veriyle doğrulanıyor.

12. Teknoloji Yığını

Katman	Teknolojiler
Uzay Mekaniği	sgp4, astropy, poliastro
Sayısal Hesaplama	numpy, scipy, pandas
Makine Öğrenmesi	scikit-learn, joblib
Backend	FastAPI, Pydantic, Uvicorn, httpx
Veritabanı	SQLite
Frontend	HTML/JavaScript, Leaflet.js, Chart.js
Test	pytest (36 deterministik test)

Sonuç

ASTM, tek bir özellik için değil, uzay trafik yönetiminin birbirine bağlı üç gerçek probleminin (çarpışma riski tespiti, kaçınma manevrası optimizasyonu, yer segmenti kapasite planlaması) bütünsel olarak ele alındığı bir sistem olarak tasarlandı. Geliştirilen sistemin gücü, şeffaf ve doğrulanabilir yapısından gelmektedir. Süreçteki tüm algoritmalar matematiksel olarak belgelenmiş, hata düzeltmeleri kayıt altına alınmış ve kritik çarpışma/kenetlenme ayrımı iddiaları, gerçek bir ISS senaryosu ve bağımsız bir kütüphane aracılığıyla çapraz doğrulanmıştır.

Ek: Erişim

Kaynak kod: <https://github.com/sebnembasak/astm-prototype/>

İletişim: <https://www.linkedin.com/in/sebnembasak1/>